

Quick Sort

```
import random

def qsort(arr, left, right):
    if left < right:
        p = get_pivot(arr, left, right)
        qsort(arr, left, p - 1)
        qsort(arr, p + 1, right)

def get_pivot(arr, left, right):
    i = left
    j = right - 1
    p = random.randint(left, right)
    arr[p], arr[right] = arr[right], arr[p]
    pivot = arr[right]
    while i < j:
        while i < right and arr[i] < pivot:
            i += 1
        while j > left and arr[j] >= pivot:
            j -= 1
        if i < j:
            arr[i], arr[j] = arr[j], arr[i]

    if arr[i] > pivot:
        arr[i], arr[right] = arr[right], arr[i]
    return i
```

Other Implementations:

```
def kth_smallest(arr, k):
    left = 0
    right = len(arr) - 1
    while left <= right:
        pivot_index = get_pivot(arr, left, right)
        if pivot_index == k - 1:
            return arr[pivot_index]
        elif pivot_index < k - 1:
            left = pivot_index + 1
        else:
            right = pivot_index - 1

def get_pivot(arr, left, right):
    i = left
    j = right - 1
    p = random.randint(left, right)
    arr[p], arr[right] = arr[right], arr[p]
    pivot = arr[right]
    while i < j:
        while i < right and arr[i] < pivot:
            i += 1
        while j > left and arr[j] >= pivot:
            j -= 1
        if i < j:
            arr[i], arr[j] = arr[j], arr[i]

    if arr[i] > pivot:
        arr[i], arr[right] = arr[right], arr[i]
    return i
```

Quick Sort Algorithm

Quick Sort is one of the different Sorting Technique which is based on the concept of Divide and Conquer, just like merge sort. But in quick sort all the heavy lifting(major work) is done while dividing the array into subarrays, while in case of merge sort, all the real work happens during merging the subarrays. In case of quick sort, the combine step does absolutely nothing.

It is also called partition-exchange sort. This algorithm divides the list into three main parts:

1. Elements less than the Pivot element
2. Pivot element(Central element)
3. Elements greater than the pivot element

Pivot element can be any element from the array, it can be the first element, the last element or any random element. In this tutorial, we will take the rightmost element or the last element as pivot.

For example: In the array {52, 37, 63, 14, 17, 8, 6, 25}, we take 25 as pivot. So after the first pass, the list will be changed like this.

{6 8 17 14 25 63 37 52}

Hence after the first pass, pivot will be set at its position, with all the elements smaller to it on its left and all the elements larger than to its right. Now 6 8 17 14 and 63 37 52 are considered as two separate subarrays, and same recursive logic will be applied on them, and we will keep doing this until the complete array is sorted.

How Quick Sorting Works?

Following are the steps involved in quick sort algorithm:

1. After selecting an element as pivot, which is the last index of the array in our case, we divide the array for the first time.
2. In quick sort, we call this partitioning. It is not simple breaking down of array into 2 subarrays, but in case of

partitioning, the array elements are so positioned that all the elements smaller than the pivot will be on the left side of the pivot and all the elements greater than the pivot will be on the right side of it.

3. And the pivot element will be at its final sorted position.
4. The elements to the left and right, may not be sorted.
5. Then we pick subarrays, elements on the left of pivot and elements on the right of pivot, and we perform partitioning on them by choosing a pivot in the subarrays.

Let's consider an array with values {9, 7, 5, 11, 12, 2, 14, 3, 10, 6}

Below, we have a pictorial representation of how quick sort will sort the given array.

Worst Case Time Complexity [Big-O]: $O(n^2)$

Best Case Time Complexity [Big-omega]: $O(n \log n)$

Average Time Complexity [Big-theta]: $O(n \log n)$

Space Complexity: $O(n \log n)$

Quicksort Complexity

Time Complexity

Best	$O(n \log n)$
------	---------------

Worst	$O(n^2)$
-------	----------

Average	$O(n \log n)$
---------	---------------

Space Complexity	$O(\log n)$
------------------	-------------

Stability	No
-----------	----

p									r
0	1	2	3	4	5	6	7	8	9
9	7	5	11	12	2	14	3	10	6



p			q						r
0	1	2	3	4	5	6	7	8	9
5	2	3	6	12	7	14	9	10	11



p	q	r		p			q		r
0	1	2	3	4	5	6	7	8	9
2	3	5	6	7	9	10	11	14	12



p, r		p, r		p		q, r		p, q	r
0	1	2	3	4	5	6	7	8	9
2	3	5	6	7	9	10	11	12	14



				p	q, r				p, r
0	1	2	3	4	5	6	7	8	9
2	3	5	6	7	9	10	11	12	14



				p, r					
0	1	2	3	4	5	6	7	8	9
2	3	5	6	7	9	10	11	12	14